

[3] Tree-Based RAG-Agent Recommendation System; A Case Study in Medical Test Data 총정리

저자: Yaofeng Yang, Chufan Huang

학회/년도: arXiv:2501.02727, 2025

분량: 약 10 페이지

역할군: (A) VAQ 동기 부여; 계층적 쿼리 패턴의 실제 사례

요약

이 논문은 RAG 기반 시스템이 의료 도메인에서 **계층적 추론**과 결합될 때 어떤 복잡한 쿼리 패턴이 필요한지를 보여준다. 단순 top-k 벡터 검색으로는 해결할 수 없는, **다단계 필터링과 집계**가 필요한 실세계 시나리오의 중요한 사례 연구이다.

의료 검사 추천 시스템의 핵심은 "환자의 증상을 보고, 어떤 진료과에 보낼지, 그리고 어떤 검사를 처방할 것인가"라는 결정이다. 이것은 단순하지 않은 **계층적 추론**을 요구한다. HiRMed (Hierarchical RAG-enhanced Medical Test Recommendation) 시스템은 트리 구조로 이를 구현하는데, 각 노드는 전문화된 RAG 에이전트이고, 상위 계층부터 하위 계층으로 점진적으로 검색 공간을 좁혀나간다.

이 구조를 SQL VAQ 로 표현하면, 각 트리 레벨에서 카디널리티가 크게 달라지는 **다단계 조인과 벡터 범위 검색의 결합**이 된다. 정확한 카디널리티 추정이 없으면, 상위 레벨의 부정확한 카디널리티가 하위 레벨로 전파되어 최악의 경우 지수적 성능 저하를 초래한다.

본 논문과의 관계; 의료 도메인에서 트리 구조의 각 노드별로 서로 다른 카디널리티 특성을 갖는 VAQ 가 발생하며, 이것이 정확한 카디널리티 추정의 필요성을 강하게 보여준다.

상세분석

3.1 해결하고자 하는 문제; 계층적 의료 추론의 필요성

의료 검사 추천 시스템에서 "환자의 증상을 보고, 어떤 진료과에 보내고, 어떤 검사를 처방할 것인가"라는 결정은 결코 단순하지 않은 **계층적 추론**을 요구한다.

현재 시스템의 문제

기존 LLM의 한계:

일반 목적(General-Purpose) LLM에 "환자가 심한 두통을 호소한다"고 입력하면:

GPT-4의 응답:
"신경학적 검사(neurological exam)와 함께,
필요시 MRI 나 CT 스캔을 추천합니다.
또한 안과 검진도 고려하세요."

문제점:

1. **과도한 추천:** 모든 가능성을 다루려고 하여 불필요한 검사까지 권장 (비용 증가)
2. **진료과 혼동:** 신경과, 안과, 이비인후과 중 어디가 1 차 진료과인지 불명확
3. **일관성 부족:** 같은 환자에 대해 시간이 지나면 다른 추천을 할 수 있음 (의료 오류 유발)

RAG를 추가해도 해결 안 되는 문제:

벡터 유사도 검색만으로는:

- "두통 케이스" 벡터를 검색해서 가장 유사한 100 개 사례를 찾은 후
- LLM에 "이 100 개 사례에서 공통 검사는 뭐야?"라고 질문

하지만 이 방식의 문제:

- 두통은 수십 가지 원인이 있는데 (뇌종양, 편두통, 안경 필요, 귀 감염 등)
- 모든 사례를 함께 고려하면 결과가 너무 다양해짐
- 특정 원인(예; 뇌종양 의심)에 특화된 검사 프로토콜을 세울 수 없음

3.2 핵심 아이디어: HiRMed (Hierarchical RAG-enhanced Medical Test Recommendation)

3 계층 트리 구조



각 노드는 **전문화된 RAG 에이전트**이며, 자신의 범위에 해당하는 지식 베이스에서만 검색·증강·생성을 수행한다.

예시: "두통" 증상 입력

Step 1 (Root 노드):

두통의 원인 분류	
- 신경과적 (45%)	
- 안과적 (20%)	
- 이비인후과적 (30%)	
- 기타 (5%)	



Step 2 (신경과 노드):

신경과 차원에서 추가 질문	
- 갑작스러운 발병? (뇌졸중 의심)	
- 반복적? (편두통 의심)	
- 국소적 신경 증상? (종양 의심)	



Step 3 (신경과 → MRI 노드):

뇌종양 선별 규약	
- MRI 기본 (모든 환자)	
- 조영제 확산 (강제)	
- 함수적 MRI (기능 손상 시)	

이중 레이어 지식 베이스

상위 레이어 (Department Knowledge Base):

증상 → 진료과 매핑 규칙

- 두통 + 시력 변화 → 안과 우선
- 두통 + 이명 → 이비인후과 우선
- 두통 + 마비 증상 → 신경과 우선

하위 레이어 (Test Knowledge Base):

진료과별 검사 상세 정보

[신경과 - MRI]

- 적응증: 뇌종양, 뇌졸중, 경련성 질환
- 금기사항: 금속 임플란트 (페이스메이커 등)
- 비용: 300,000 원
- 소요시간: 40 분
- 진단 가능성: 뇌종양 95%, 편두통 10%

메모리 증강 추론

이전 진단 이력을 메모리에 유지하여:

같은 환자 (ID: P12345)

- 1 차 방문: 두통 → 신경과 의뢰 → MRI 처방
- 2 차 방문: 지속적 두통 → 신경과 (이미 방문함) → 추가 검사 결정
 - 메모리에서 "이 환자는 신경과 경험 있음"을 참조
 - 동일 검사 반복 회피 가능

3.3 성능 결과

의료 검사 추천에 대한 평가:

평가 지표	단일 LLM	HiRMed	개선도
커버리지 (관련 검사를 빠뜨리지 않음)	78.9%	92.3%	+13.4%
정확도 (추천한 검사가 실제로 적절)	71.2%	88.7%	+17.5%
비용 효율성 (불필요한 검사 회피율)	45%	78%	+33%
일관성 (같은 환자의 반복 방문 시 동일 추천)	62%	94%	+32%

3.4 트리 구조의 카디널리티 특성

Root 레벨의 카디널리티

"두통" 증상의 환자 풀:

전체 환자 DB: 1,000,000 명
두통으로 내원한 환자: 50,000 명 (선택도 5%)

진료과별 분류:

- 신경과 (진료과 A): 22,500 명 (45%)
- 안과 (진료과 B): 10,000 명 (20%)
- 이비인후과 (진료과 C): 15,000 명 (30%)
- 기타 (진료과 D): 2,500 명 (5%)

검사 노드의 카디널리티

신경과 내에서 더 세밀한 분류:

신경과 환자 22,500 명

MRI 추천:

- 급성 신경 증상: 5,000 명 → MRI 필수 (100% 선택도)
 - 만성 두통 (3 개월 이상): 8,000 명 → MRI 선택적 (60% 선택도)
 - 편두통 (빈번하지 않음): 9,500 명 → MRI 불필요 (5% 선택도)
- 전체 신경과 환자 중 MRI 추천율: $(5000 + 4800 + 475) / 22500 = 28\%$

3.5 VAQ 로의 변환

HiRMed 의 검사 추천 로직을 SQL 로 표현하면:

```
-- 1 단계: 증상 기반 진료과 후보 검색
SELECT dept_id, dept_name, matching_score
FROM departments
WHERE vector_distance(dept.symptom_profile, query_symptom_embedding) < 0.5
ORDER BY matching_score DESC;

-- 2 단계: 각 진료과 내 구체적 검사 추천
SELECT test_id, test_name, recommendation_score
FROM medical_tests
WHERE
  -- 벡터 유사도: 환자 증상과 검사의 적응증 일치
  vector_distance(test.indication_embedding, query_symptom_embedding) < 0.4

  -- 메타데이터 필터: 환자 프로필과의 호환성
  AND patient_age BETWEEN test.recommended_age_min AND test.recommended_age_max
  AND test.contraindication NOT IN (patient_medical_history)

  -- 진료과 필터: 1 단계에서 선정된 진료과만
  AND test.department_id IN (SELECT dept_id FROM step1_result)

  -- 비용 제약: 보험 적용 검사만
  AND test.insurance_covered = TRUE

ORDER BY recommendation_score DESC;

-- 3 단계: 환자 이력 고려 (메모리 증강)
SELECT test_id
FROM step2_result
WHERE
  -- 같은 환자가 최근 12 개월 내 받은 검사 제외
  test_id NOT IN (
    SELECT test_id FROM patient_history
    WHERE patient_id = :patient_id
    AND test_date > NOW() - INTERVAL '12 months'
  );
```

이 SQL 쿼리에서:

- **1 단계:** 벡터 검색 결과 카디널리티 = 3~5 진료과
- **2 단계:** 각 진료과별 검사 후보 = 수십~수백 개 검사
- **3 단계:** 최종 필터링 후 = 3~10 개 검사

카디널리티의 급격한 변화:

- 1 단계 결과 3 → 2 단계 중간 결과 300 → 3 단계 최종 8
- 이런 급격한 변화에는 정적 통계가 무용지물

3.6 정확한 카디널리티의 가치

카디널리티 추정 오류의 영향

시나리오 A: 카디널리티 과소 추정

실제 카디널리티: 500 개 검사
추정 카디널리티: 10 개 검사 (추정 오류)

옵티마이저의 판단:

- "1 단계 결과를 먼저 조인하고, 그 다음 벡터 검색"
- 실행 계획: JOIN (3 개 진료과) JOIN FILTER (vector search)
- 예상 비용: 낮음 ($3 \times$ 처리 비용)

실제 실행:

- 3 개 진료과 조인으로 22,500 건 나옴
- 각 22,500 건에 대해 벡터 검색 (비싼 연산) 수행
- 실제 비용: 매우 높음 ($22,500 \times$ 벡터 검색 비용)

시나리오 B: 카디널리티 과다 추정

실제 카디널리티: 50 개 검사
추정 카디널리티: 500 개 검사 (추정 오류)

옵티마이저의 판단:

- "벡터 검색을 먼저 수행, 그 결과만 조인"
- 실행 계획: VECTOR_SEARCH (500 개 후보) → JOIN 필터
- 예상 비용: 높음 ($500 \times$ 처리 비용)

실제 실행:

- 벡터 검색 수행 (실제로는 50 개만 해당)
- 하지만 이미 500 개 후보 검색 비용이 발생함
- 불필요한 계산 450 개 분 낭비

정확한 카디널리티로 얻는 이득

각 단계에서:

- 1 단계 결과가 3 개 → 조인 비용 최소화
- 2 단계 중간 결과가 300 개 → 해시 조인 선택 (Nested Loop 피함)
- 3 단계 필터가 8 개 → Sort 생략, Limit 최적화

전체 성능:

- 부정확한 계획: 응답시간 10~20 초 (비용 초과)
- 정확한 계획: 응답시간 1~2 초 (10 배 이상 개선)

3.7 본 논문과의 관계

HiRMed의 각 노드에서 수행하는 RAG 검색은:

벡터 유사도 (증상 임베딩 매칭)
 + 메타데이터 필터 (진료과 코드, 연령, 금기사항 등)
 + 집계 (우선순위 랭킹)
 + 과거 이력 제외

을 결합하는 **복합 VAQ**이다.

이 과정이 SQL로 표현되면 **다단계 조인과 벡터 범위 검색**의 결합이 되며, 특히:

- 트리의 각 레벨에서 카디널리티가 **크게 달라짐** (Root 50,000 → 검사 500 → 최종 8)
- 상위 레벨의 카디널리티 오류가 하위 레벨로 전파됨 (지수적 성능 저하)
- 정적 통계로는 이 변동성을 포착하기 어려움

따라서 Exqutor의 **동적 카디널리티 추정**이 특히 유용한 시나리오이다.

추가 제기 문제

1. 트리 구조의 고정성:

- 현재 진료과 분류는 고정되어 있다.
- 새로운 진료과(예; AI 기반 정신건강 상담)가 추가되면 트리 재구성 필요
- 동적으로 트리를 조정하는 메커니즘이 필요하지 않을까?

1. 노드 간 쿼리 최적화 부재:

- 각 노드의 RAG 검색이 독립적으로 수행됨
- 예; Root 에서 찾은 진료과 정보를 하위 노드에서 재활용할 수 있는데 활용 안 함
- 노드 간 중간 결과 공유 (캐싱)로 성능 개선 가능 → Exqutor 의 아이디어와 유사

1. 멀티모달 의료 데이터 미지원:

- 현재는 텍스트 증상만 고려
- X-ray 이미지나 혈액 검사 수치 같은 구조화 데이터를 함께 고려하면?
- 더 정교한 추천이 가능하지만 복잡도 급증 → 카디널리티 추정 더욱 어려워짐

1. 피드백 루프 부재:

- 추천한 검사가 실제로 효과적이었는가를 시스템이 학습하지 않음
- 시간에 따라 진료 가이드라인이 변하는데 (의료 지식 업데이트)
- 시스템이 자동으로 적응하는 메커니즘이 필요
- Exqutor 의 런타임 적응적 샘플링이 이 문제의 해답이 될 수 있을까?